

synaptic 技术手册

内部架构与自定义扩展

模板开发团队

CTAN 社区

电子邮箱: synaptic@example.com

摘要: 本文档面向包开发者和高级用户，详细说明 synaptic 的内部数据结构、标签与样式自定义接口、定理环境扩展以及字体加载与回退机制。同时通过实例展示如何在不修改核心文件的情况下完全定制外观。

关键词: synaptic, 技术手册, 内部实现, 自定义

arXiv 电子文献: [2605.54321](https://arxiv.org/abs/2605.54321)

献给所有热爱 $\TeX$ 排版艺术的探索者。

Contents

1	架构概览	1
2	标签与样式的自定义	1
3	定理环境的底层构造	1
4	字体加载与回退逻辑	2
5	颜色方案的内部实现	2
6	精细断行的实现	2

1 架构概览

`synaptic` 全部以 `expl3` 编写，内部函数前缀为 `\__synaptic_` 或 `\g__synaptic_`。键值选项通过 `l3keys2e` 处理并在早期执行。

核心数据容器：

- `\g__synaptic_labels_prop` — 标签文本（如“摘要”“定理”等）
- `\g__synaptic_styles_prop` — 样式代码（如定理头字体）
- `\g__synaptic_meta_prop` — 标题页元数据
- `\g__synaptic_authors_seq`、`\g__synaptic_affils_seq`、`\g__synaptic_emails_seq` — 作者、单位、邮箱列表

这些容器均通过 `\prop_gput:Nnn` 和 `\seq_gput_right:Nn` 填充，保证了扩展性。

2 标签与样式的自定义

用户可在导言区使用以下命令覆盖任何标签或样式：

```
\synaptic_register_label:nn { abstract } { 摘要 }
\synaptic_register_style:nn { thmhead }
{ \normalfont\bfseries\color{blue} }
```

支持的标签键：`abstract`、`keywords`、`arxiv`、`default_header`、`acknowledgments`、`proof`、`thm`、`lemma`、`prop`、`def`、`email`。

样式键：`thmhead`（定理头）、`thmnote`（定理注释）、`label`（标题页标签字体）、`thmbody`（定理正文）。

注意 2.1. 自定义标签或样式应在 `\maketitle` 之前完成，以确保标题页和定理环境使用正确的名称和字体。

3 定理环境的底层构造

预定义的定理样式名为 `syn_thm_modern_style`，它使用 `\declaretheoremstyle` 定义。所有定理均通过 `\declaretheorem` 基于该样式创建，且默认与 `theorem` 计数器兄弟关联。

```
\declaretheorem[
  style = syn_thm_modern_style,
  name  = \synaptic_get_label:n { lemma },
  sibling = theorem
]{lemma}
```

`\declarecustomtheorem` 宏是对上述过程的封装，便于用户快速添加新环境。

记号 3.1. 本文档中我们自定义了 `notation` 和 `caution` 环境，以展示扩展的简洁性。

4 字体加载与回退逻辑

包使用 `\IfFontExistsTF` 检测字体是否存在，并提供显式回退。例如，主罗马字体优先选择 XCharter，若缺失则回退至 Latin Modern Roman，并给出警告。中文模式通过 `\ctex` 加载，选项 `scheme=plain` 可避免与 `amsthm` 冲突。

5 颜色方案的内部实现

所有颜色均继承自 `syncolor`，其派生颜色通过 `\colorlet` 计算，例如：

```
\colorlet{syncolor-light}{syncolor!40}
\colorlet{syncolor-rule}{gray!40}
```

若要完全自定义颜色，只需在文档前导区重新定义 `syncolor` 及其派生色。

6 精细断行的实现

当 `fine-breaking=true` 时，包会设置：

```
\emergencystretch=4em
\tolerance=4000
\widowpenalty=10000
\clubpenalty=10000
```

并启用 `\raggedbottom`。这些参数在不破坏整体对齐的前提下有效减少了断行问题。

致谢

本文档由 synaptic 团队编写，欢迎通过 CTAN 提交改进建议。